

Uniswap Protocol Fees V4 Adapter Audit



Uniswap

May 20, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents	2
Summary	3
Scope	4
System Overview	5
V4FeeAdapter	5
V4FeePolicy	6
IFeeClassifiedHook and HookFeeFlags	6
Security Model and Trust Assumptions	7
Privileged Roles	7
Trust Assumptions	7
Low Severity	9
L-01 Ambiguous feeValue == 0 in Fee Update Events	9
L-02 Truncated Multiplier Result Silently Skips familyDefaults Fallback	9
Notes & Additional Information	10
N-01 Unused State Variable	10
N-02 Duplicated Fee Encoding and Decoding Logic	11
N-03 Floating Pragma	11
N-04 Gas Optimizations	11
N-05 Inconsistent Interface for Unsetting Hook Family	12
N-06 Code Clarity	12
N-07 Unused Imports	12
N-08 Hook Self-Report Staticcall Lacks Returndata Size Bound	13
N-09 Flag Rules Ordering Could Be Enforced Onchain	13
Conclusion	14
Appendix	15
Issue Classification	15

Summary

Type	DeFi	Total Issues	11 (8 resolved)
Timeline	From 2026-05-11 To 2026-05-20	Critical Severity Issues	0 (0 resolved)
Languages	Solidity	High Severity Issues	0 (0 resolved)
		Medium Severity Issues	0 (0 resolved)
		Low Severity Issues	2 (2 resolved)
		Notes & Additional Information	9 (6 resolved)
		Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of [pull request #133](#) on the [Uniswap/protocol-fees](#) repository at the [85ce29f](#) commit.

In scope were the following files:

```
src
├── feeAdapters
│   ├── V4FeeAdapter.sol
│   └── V4FeePolicy.sol
├── interfaces
│   ├── IFeeClassifiedHook.sol
│   ├── IV4FeeAdapter.sol
│   └── IV4FeePolicy.sol
└── libraries
    └── HookFeeFlags.sol
```

System Overview

The system in scope adds a Uniswap V4 protocol fee controller, the `V4FeeAdapter`, paired with a replaceable fee resolution backend, the `V4FeePolicy`. Together they let governance compute and push per-pool protocol fees into the V4 `PoolManager`, sweep accrued fees to a fixed recipient, and self-report optional pool characteristics from hooks. The two contracts are deployed alongside each other, both exposing independent owner roles and a `feeSetter` role for day-to-day configuration. Fees are resolved per `PoolKey` and pushed lazily through a permissionless trigger, never read by the `PoolManager` at pool initialization.

V4FeeAdapter

The `V4FeeAdapter` is the long-lived contract registered with the `PoolManager` as `protocolFeeController`. It owns the privilege to call `setProtocolFee(key, fee)` and `collectProtocolFees(recipient, currency, amount)` on the manager, and it is the only on-chain piece that interacts with the V4 fee surface. Its responsibilities are:

- Resolve the effective fee for a pool through a three-tier waterfall (per-pool override, then policy, then zero) in `getFee`.
- Expose permissionless `triggerFeeUpdate` and `batchTriggerFeeUpdate` so anyone can push the currently resolved fee into the `PoolManager` for one or more pools, after a sanity check that `slot0.sqrtPriceX96` is non-zero.
- Expose permissionless `collect`, which forwards every requested currency from the `PoolManager` to the immutable `TOKEN_JAR` recipient and emits one event per item.
- Provide owner-gated `setPolicy` and `setFeeSetter` for governance rotation, and feeSetter-gated `setPoolOverride` and `clearPoolOverride` for pinning a specific fee on a specific pool.
- Encode an explicit "zero protocol fee" as a `ZERO_FEE_SENTINEL` value (`type(uint24).max`) so that an unset storage slot remains distinguishable from a configured zero.

V4FeePolicy

The `V4FeePolicy` is the hot-swappable computation backend invoked from `V4FeeAdapter.getFee`. It carries no pool-level state and does not interact with the `PoolManager` itself. Its `computeFee(PoolKey)` entry point selects between two resolution paths based on the hook address bits and the pool's `key.fee`:

- **Path A (StaticNativeMath)** runs for vanilla pools (no `*_RETURNS_DELTA` flags in the hook address, and a non-dynamic `key.fee`). The policy either returns a configured `pairFees` value, or walks a piecewise-linear bucket schedule keyed off `key.fee`, producing $\alpha + \beta * \text{delta} / 1_000_000$ per direction and clamping the result to `MAX_PROTOCOL_FEE`.
- **Path B (Classified)** runs for hooks that take swap or liquidity surpluses, or for pools with the dynamic fee sentinel `0x800000`. The policy resolves a hook family (either a governance pin via `setHookFamily` or via `_flagRules` matched against the hook's optional `protocolFeeFlags()` self-report), then layers a per-family multiplier on the pair fee, or returns a per-family default, or finally falls through to `defaultFee`.

All policy configuration is gated by a `feeSetter` role independent of the adapter's. Setters that replace arrays (`setFeeBuckets`, `setFlagRules`) operate atomically.

IFeeClassifiedHook and HookFeeFlags

The `IFeeClassifiedHook` interface defines a single optional view function, `protocolFeeFlags()`, that hooks may implement to advertise a `uint256` bitmask of behavioral properties (for example `TAKES_SWAP_SURPLUS`, `USES_DYNAMIC_FEE`, `STABLE_PAIR`). The corresponding bit constants live in the `HookFeeFlags` library. The policy reads these flags through a `staticcall` capped at 30,000 gas and uses them only as one input to family resolution. The interface is opt-in, untrusted, and intentionally placed in `src/interfaces` rather than `v4-core` so it can evolve without a `v4-core` dependency.

Security Model and Trust Assumptions

The design treats the adapter as the long-lived, trusted boundary holding the `protocolFeeController` privilege, and treats the policy as a replaceable computation module behind it. Permissionless functions on the adapter cannot push a fee above `MAX_PROTOCOL_FEE` (1000 pips per direction, where one pip equals 1/1,000,000, i.e., 0.1% of swap volume) because the `PoolManager` re-validates every pushed value, and they cannot redirect collected fees because the recipient is the `immutable TOKEN_JAR`. Hook contracts are explicitly modeled as adversarial, with the worst-case outcome being a self-downgrade to `defaultFee`.

Privileged Roles

- Adapter `owner` (solmate `Owned`, single-step): can call `setPolicy` and `setFeeSetter`. Setting `policy = address(0)` is the intended kill switch and short-circuits `getFee` to zero.
- Adapter `feeSetter`: can call `setPoolOverride` and `clearPoolOverride`, bounded per direction by `MAX_PROTOCOL_FEE`.
- Policy `owner` (independent on-chain from the adapter `owner`): can call `setFeeSetter` on the policy.
- Policy `feeSetter` (independent from the adapter `feeSetter`): can call every `setX` and `clearX` on the policy (hook families, flag rules, fee buckets, default fee, family defaults, family multipliers, pair fees). All array replacements are atomic.
- `PoolManager owner` (out of scope): can call `setProtocolFeeController(other)` and revoke the adapter's privilege at any time.

Trust Assumptions

- **Actor Honesty.** Adapter and policy `owner` and `feeSetter` are trusted to act in the protocol's interest. The two `owner` roles can be the same address or different addresses; on-chain equality is not enforced and the deploy script is responsible for coordinated rotation. Single-step `Owned` is accepted as the intended privilege model.

- **External Data Integrity.** The Uniswap V4 `PoolManager` (audited separately, pinned at commit `59d3ecf`) is treated as a trusted black box. It is responsible for re-validating every pushed fee through `isValidProtocolFee`, enforcing the `ProtocolFeeCurrencySynced` revert on collect during an active `sync`, and applying the protocol fee on top of the LP fee through `ProtocolFeeLibrary.calculateSwapFee`. Solmate `Owned` is similarly trusted. Hooks at `key.hooks` are explicitly **not trusted**, may return arbitrary bytes from `protocolFeeFlags()`, may revert or run out of gas, and may be upgradeable behind their immutable address.
- **Secure Runtime.** The contracts compile against `^0.8.26` and assume an EVM with EIP-1153 transient storage (Cancun or later), since the `PoolManager`'s `sync` / `settle` flow relies on it. The deploy script is responsible for setting `TOKEN_JAR` to a real recipient, for registering the adapter as `protocolFeeController` on the target `PoolManager`, and for the post-deploy `setFeeSetter` calls that activate configuration. Both contracts begin life with `feeSetter == address(0)`, which safely freezes all configuration setters until the owner initializes the role.
- **Scope of Responsibility.** The adapter does not maintain a registry of live pools and does not auto-push fees after a configuration change. Synchronizing `slot0.protocolFee` with the latest resolved fee is the responsibility of the caller of `triggerFeeUpdate` or `batchTriggerFeeUpdate`. The harm from a stale on-chain fee is bounded by `MAX_PROTOCOL_FEE`. Per assumption, hook flag-rule configuration must only point at families whose effective fee is at least `defaultFee`, so a hook that lies about its flags or self-downgrades by reverting or running out of gas cannot reduce its protocol fee below `defaultFee`. The behavior of the `TokenJar` after receiving funds, the internals of the V2 and V3 adapters, the periphery, deploy scripts, and tests are outside the scope of this engagement.

Low Severity

L-01 Ambiguous `feeValue == 0` in Fee Update Events

The `V4FeeAdapter` and `V4FeePolicy` contracts store explicit zero fees using a `sentinel value`. However, update events still emit `feeValue = 0` for `both` explicit-zero overrides and remove/unset actions, which can hinder off-chain state indexing.

Additionally, the `_encodeFee` (1, 2) function and the `FamilyDefaultUpdated`, `PairFeeUpdated`, and `DefaultFeeUpdated` events incorrectly document `feeValue == 0` as 'override removed', even though it can also indicate an explicit zero assignment.

Consider making fee update events unambiguous by emitting either the encoded storage value or distinct "set" and "cleared" events.

Update: Resolved at commit [e38d513](#).

L-02 Truncated Multiplier Result Silently Skips `familyDefaults` Fallback

On the Classified path, `computeFee` commits to the multiplier branch as soon as both `pairFees[ph]` and `familyMultiplierPips[family]` are non-zero. It returns `_applyMultiplier(_decodeFee(pairFee), multiplier)` unconditionally, and only consults `familyDefaults[family]` when one of those inputs is zero.

`_applyMultiplier` scales each 12-bit directional component with integer division:

```
fee0 = baseFee0 * multiplier / 1_000_000
fee1 = baseFee1 * multiplier / 1_000_000
```

So any non-zero component whose `component * multiplier < 1_000_000` rounds to zero. The caller does not distinguish this rounded-zero result from an intentional zero fee, so the family default is bypassed.

A realistic configuration:

- `pairFees[(USDC, WETH)] = (100 << 12) | 100` (100 pips per direction)
- `familyMultiplierPips[5] = 9_999`
- `familyDefaults[5] = (50 << 12) | 50` (intended fallback floor)

The expected output is either 1 pip per direction, or the family default if the operator expects the default to backstop rounding. The actual output is:

```
100 * 9_999 / 1_000_000 = 0
```

so `computeFee` returns `0` for both directions. `StaticNativeMath` pools with the same pair fee return the pair fee directly, because that path never applies the family multiplier.

This is not externally exploitable under the current trust model, since `feeSetter` controls the configuration. The issue is the silent discontinuity: `multiplier == 0` falls through to `familyDefaults`, while any positive multiplier that truncates to zero short-circuits past it.

Consider either documenting this behavior or adjusting the Classified path so that a non-zero pair fee rounding to zero falls through to the family default, while explicit zero pair fees remain preserved via the [sentinel value](#).

Update: Resolved at commit [043f512](#).

Notes & Additional Information

N-01 Unused State Variable

Within the `V4FeePolicy` contract, the `POOL_MANAGER state variable` is unused.

To improve the overall clarity and intent of the codebase, consider removing any unused state variables.

Update: Acknowledged, not resolved.

N-02 Duplicated Fee Encoding and Decoding Logic

The `V4FeePolicy` and `V4FeeAdapter` contracts share the same encoding and decoding scheme for protocol fees, in which the value `0` is stored as `ZERO_FEE_SENTINEL = type(uint24).max` to distinguish it from uninitialized storage slots. However, the `ZERO_FEE_SENTINEL` constant and the `_encodeFee` and `_decodeFee` functions are duplicated across both contracts.

Consider extracting the duplicated constant and functions into a reusable library, which could also include the `_validateFee` function that is duplicated as well.

Update: Acknowledged, not resolved.

N-03 Floating Pragma

Pragma directives should be fixed for deployable contracts to clearly identify the Solidity version with which the contracts will be compiled. Throughout the codebase, `V4FeePolicy.sol` and `V4FeeAdapter.sol` use the `solidity ^0.8.26` floating pragma directive.

Consider using fixed pragma directives for all deployable contracts.

Update: Resolved at commit [fdc72f4](#).

N-04 Gas Optimizations

The following opportunities to optimize gas usage were identified throughout the codebase:

- The `batchTriggerFeeUpdate` function in `V4FeeAdapter` does not cache the array length, unlike the `collect` function.
- The `loop condition` in `_computeStaticNativeMathFee` could be changed to `i > 1`, since `i = 1` would assign `bucket = _feeBuckets[0]`, which is already [the default](#).
- The `policy` state variable could be cached within `getFee` to avoid reading storage [twice](#).
- In `_resolveFamily`, `rule.requiredFlags` could be cached to avoid a [duplicate read](#) from storage.

Consider addressing the points outlined above to reduce operational expenses for both users and administrators.

Update: Resolved at commit [604001f](#).

N-05 Inconsistent Interface for Unsetting Hook Family

The `IV4FeePolicy` interface defines separate functions for setting and clearing most configuration parameters. The `setHookFamily` function is an exception, as it is designed to handle both setting and clearing the family ID of a hook.

Consider adding a `clearHookFamily` function to maintain a consistent interface across all operations.

Update: Acknowledged, not resolved.

N-06 Code Clarity

The following opportunities to improve code clarity were identified in the `V4FeePolicy` contract:

- In lines [286](#) and [297](#), the `>=` operator could be applied without unwrapping the custom type, since the operator is [overloaded](#) in v4-core's `Currency.sol`.
- In [line 48](#), `MULTIPLIER_DENOMINATOR` could be initialized or replaced with `ProtocolFeeLibrary.PIPS_DENOMINATOR`.
- In [line 57](#), `MAX_BETA_PIPS` could be initialized to `ProtocolFeeLibrary.PIPS_DENOMINATOR * ProtocolFeeLibrary.MAX_PROTOCOL_FEE`.

Consider applying the above recommendations to improve the clarity and readability of the codebase.

Update: Resolved at commit [a0d2c3e](#).

N-07 Unused Imports

The `PoolId` and `PoolIdLibrary` [imports](#) in `V4FeePolicy` are unused.

Consider removing unused imports to improve the readability of the codebase.

Update: Resolved at commit [604001f](#).

N-08 Hook Self-Report Staticcall Lacks Returndata Size Bound

The `staticcall` in `_resolveFamily` caps the callee at `SELF_REPORT_GAS_LIMIT = 30000` but does not bound the returndata size copied back into caller memory. Solidity lowers the high-level call to a STATICCALL with `retLength = 0`, followed by an unbounded `returndatacopy` of `returndatasize()`. A hook that uses its budget to reach a roughly 100 KB memory high-water mark and returns all of it can force the caller to pay approximately 40,000 extra gas for memory expansion and copying, roughly doubling the practical per-call cost on top of the reserved 30,000.

The subsequent `abi.decode(ret, (uint256))` consumes only the first 32 bytes of `ret`, so anything beyond is paid-for waste even with an honest hook. Consider replacing the high-level staticcall with an inline-assembly STATICCALL that specifies `retLength = 32` directly, which bounds the caller's returndata handling to a small constant regardless of `returndatasize()` or future memory-expansion gas pricing.

Update: Resolved at commit [fa0c270](#).

N-09 Flag Rules Ordering Could Be Enforced Onchain

Governance-set `flag rules` classify hooks into families based on their `self-reported flags`. Rules are checked in order, and the first rule whose `requiredFlags` are all present in the hook's self-reported flags wins, so more specific patterns must come first. Currently, the `V4FeePolicy` contract relies on Governance to specify the flag rules in the correct order.

Consider enforcing this ordering within `setFlagRules` by requiring `rules` to be sorted by non-increasing "number of bits set" on the `requiredFlags` property. This way, more specific rules (i.e., with more bits set) are guaranteed to come first. An efficient reference implementation for counting set bits can be found in Solady's `LibBit` library.

Update: Resolved at commit [cd724fa](#).

Conclusion

The `V4FeeAdapter` and `V4FeePolicy` contracts together implement a governance-controlled protocol fee surface for Uniswap V4. The adapter is registered as the `protocolFeeController` on the `PoolManager` and exposes permissionless entry points to push resolved fees into the `PoolManager` and sweep accrued protocol fees to a fixed recipient, alongside owner- and feeSetter-gated configuration. The policy acts as a hot-swappable computation backend that resolves fees per `PoolKey` through two paths: a static path that walks a piecewise-linear bucket schedule, and a classified path that, for hooks with dynamic fees and/or custom accounting, layers per-family multipliers on top of a pair fee.

The findings identified during this audit primarily focused on improving the clarity, consistency, and efficiency of the codebase. A recommendation was also provided to strengthen on-chain validation of configuration parameters and reduce reliance on governance. The Uniswap team is commended for their responsiveness and transparency throughout the review process. Their readiness to provide detailed technical explanations and clarify design decisions greatly facilitated the assessment and demonstrated a strong commitment to making their product as secure as possible.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.